



SW개발/HW제작 설계서

자연어 처리 (NLP) 와 머신러닝을 이용한
리뷰 데이터 분석 및 사용자 경험 분석

2021. 11. 03

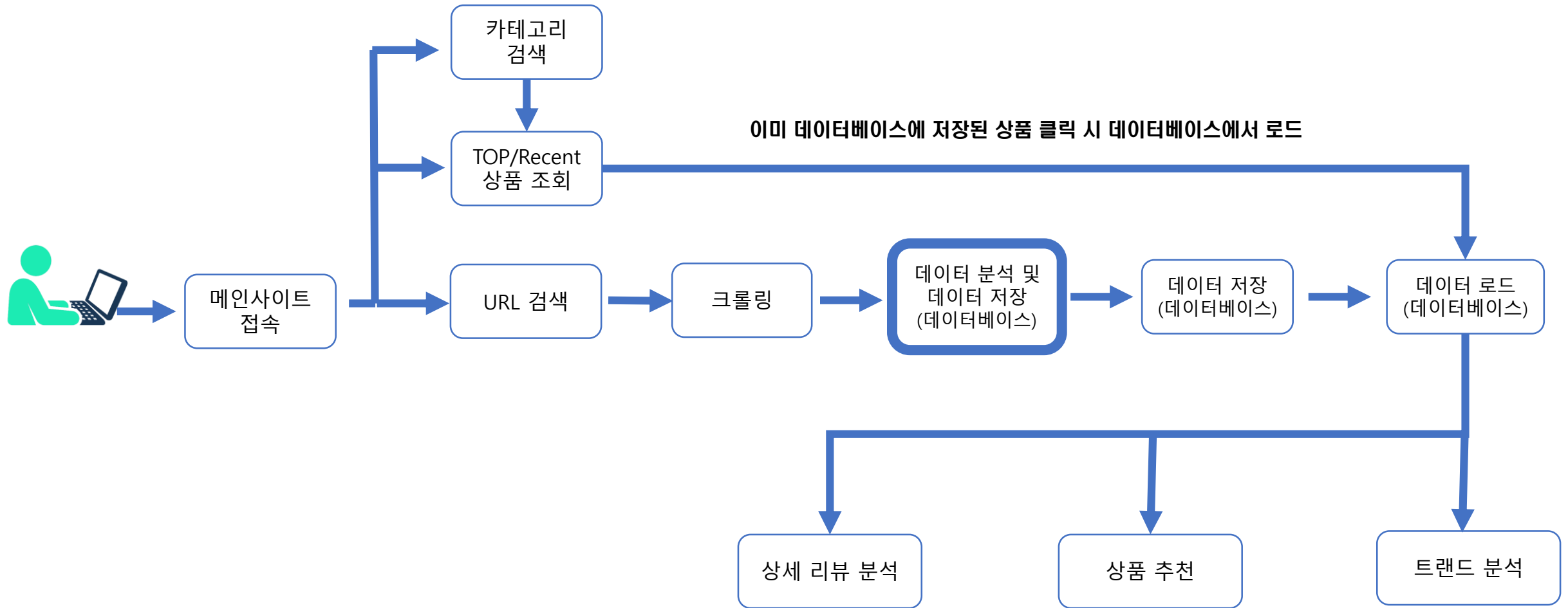
| 요구사항 정의서

구분	기능	설명
S/W	크롤링	분석하고 싶은 상품의 URL을 입력 받아, 해당 상품의 URL을 크롤링 하여 가져옵니다.
	감성분석(XAI)	크롤링 한 리뷰데이터를 감성분석 모델을 통해 분석합니다. 이 과정 중, 어떠한 단어의 영향으로 혹은 문맥의 영향으로 긍정 혹은 부정으로 판단하였는지, 설명가능한 AI 모델을 제작합니다.
	절대평점	쇼핑몰의 평점이 아닌, 감성분석을 통해 얻어낸 값을 통해 해당 상품의 평점을 매깁니다.
	키워드화	상품 리뷰데이터를 분석하여 해당 상품의 핵심 키워드 및 해당 키워드의 감성분석 결과를 추출합니다.
	리뷰 요약	각 키워드의 감성분석 결과를 바탕으로 필터링된 리뷰의 임베딩 값(리뷰 속의 단어들의 평균 임베딩 값을 통해 각 리뷰를 임베딩 값으로 표현)을 평균 내어 이와 가장 비슷한 값을 가진 리뷰를 코사인 유사도를 통하여 추출합니다.
	리뷰기반 상품추천	리뷰를 기반으로 추출된 키워드를 사용자가 선택하며, 선택된 키워드에 적합한 상품을 추천합니다.
	트렌드 분석	리뷰의 특징 키워드, 빈도 수, 작성 일자 등을 분석하여 카테고리 별 시각화를 합니다.
	웹사이트 / 클라우드 서버 구축	웹사이트 구축을 통해, 완성된 모델을 웹사이트에 적용하고, 실시간 리뷰 분석 결과를 시각화하는 MLops 전반적인 과정을 설계합니다.

| 유즈케이스

카테고리를 입력 시, 해당 카테고리의 Top/Recent 상품 조회

이미 데이터베이스에 저장된 상품 클릭 시 데이터베이스에서 로드



| 서비스 구성도 - 서비스 시나리오



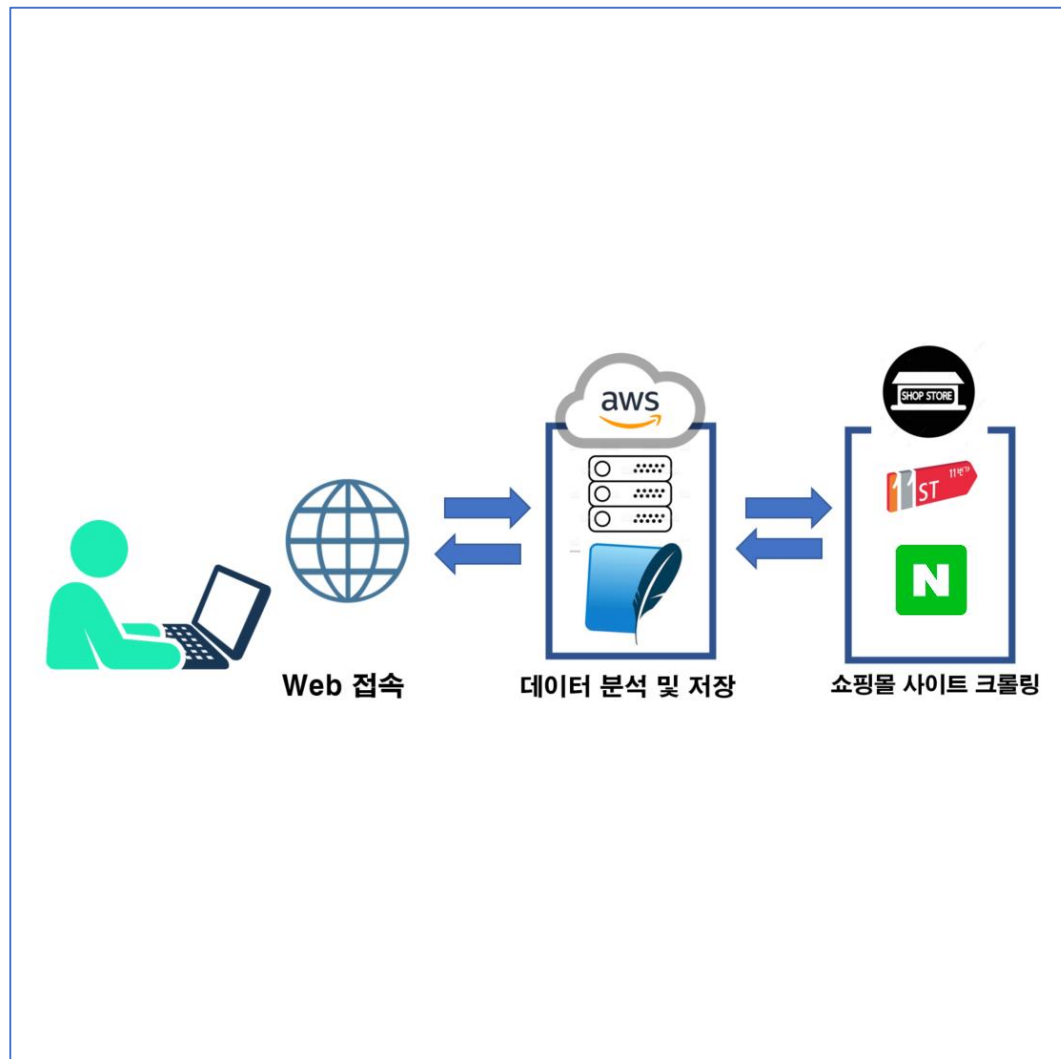
Front

1. 사용자가 웹 페이지에 접속하면 디자인된 HTML 제공
2. 사용자 이벤트에 따라 적절한 액션 또는 반응 제공
3. 데이터 입력(URL 정보 or 카테고리 정보)이 이루어지면 해당 데이터를 서버로 전송
4. 전처리나 모델 적용이 완료되어 서버로부터 반환된 주요 기능 및 통계 결과 출력

Back

1. 입력 받은 URL을 이용하여, 쇼핑몰 사이트로 부터 리뷰데이터를 크롤링 합니다.
2. 수집된 리뷰 데이터를 데이터베이스에 저장 후, 데이터 분석을 실시합니다.
3. 입력 받은 URL이 이미 데이터베이스에 있을 경우, 해당 상품에 대한 내용을 출력합니다.
4. 카테고리를 입력 받을 경우, 해당 카테고리에 해당하는 상품들을 데이터 베이스에서 출력합니다.

| 서비스 흐름도



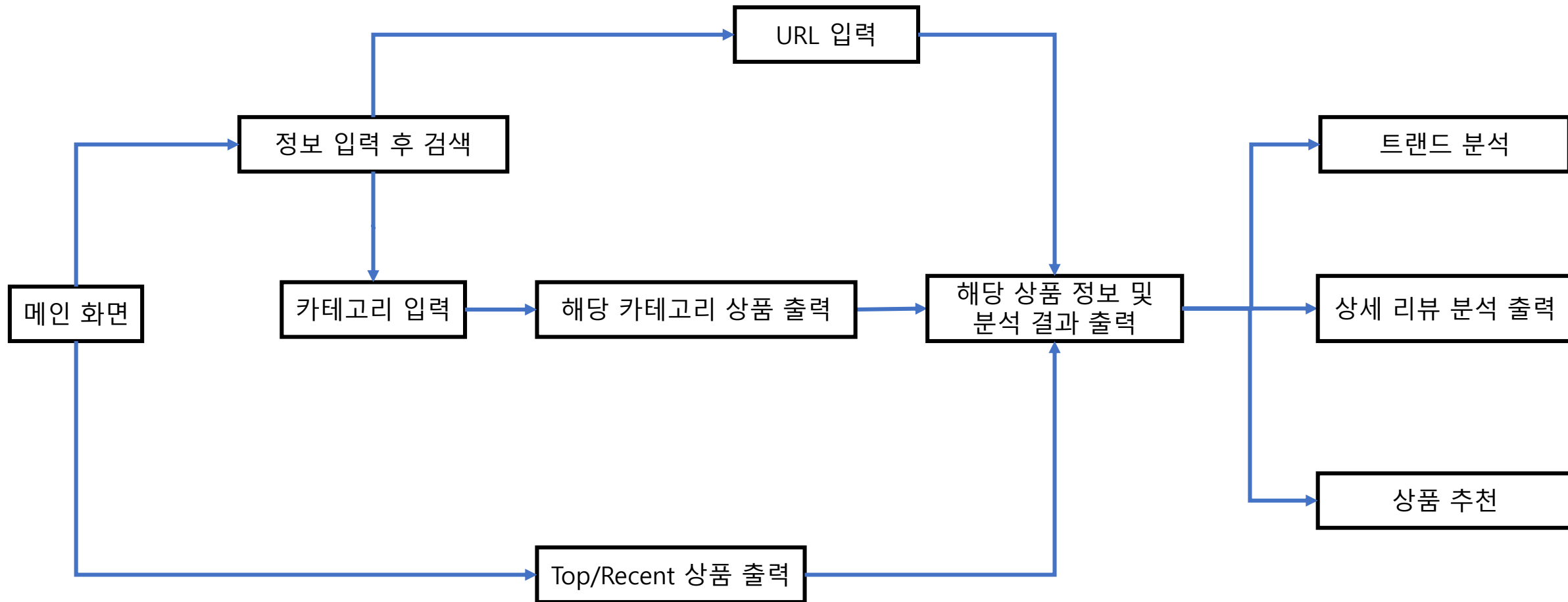
- 리뷰데이터 분석

- 1) 사용자로부터 검색하고 싶은 상품의 URL을 입력 받습니다.
- 2) 입력 받은 URL을 이용하여 서버를 통해 해당 상품의 리뷰를 크롤링합니다.
- 3) 수집된 리뷰 데이터를 준비된 모델을 이용하여, 감성 분석, 문장 요약, 키워드화 등 분석 합니다.
- 4) 데이터베이스에 저장된 모델을 로드 하여 화면에 출력합니다.

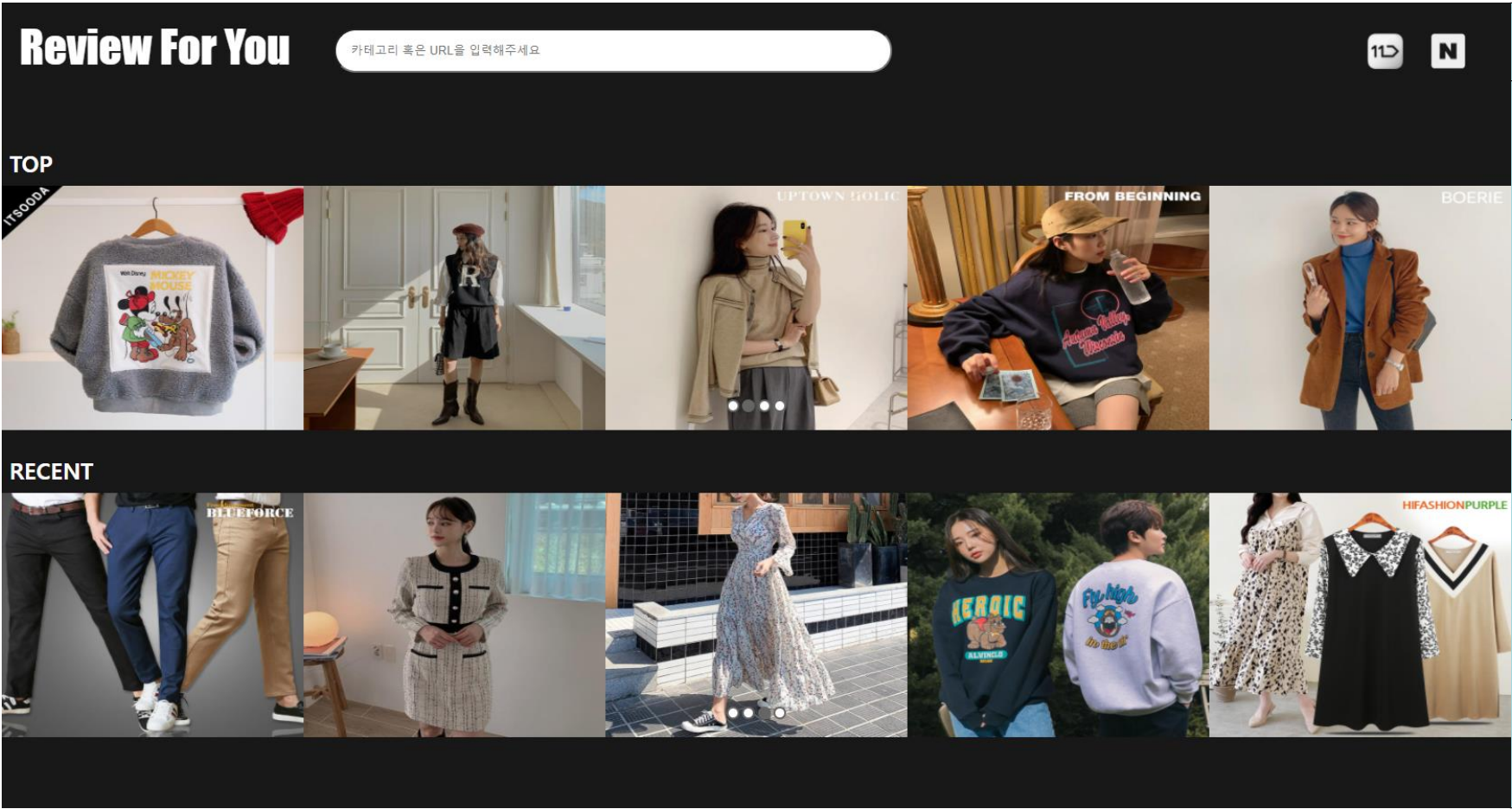
- 상세 분석

- 1) 사용자로부터 분석하고 싶은 상세 리뷰 or 트렌드 분석을 입력 받습니다.
- 2-1) 상세 리뷰일 경우, 해당 리뷰의 상세 분석을 출력합니다.
- 2-2) 트렌드 분석일 경우, 클릭된 상품의 카테고리를 분석 하여, 시각화로 나타냅니다.

| 메뉴 구성도

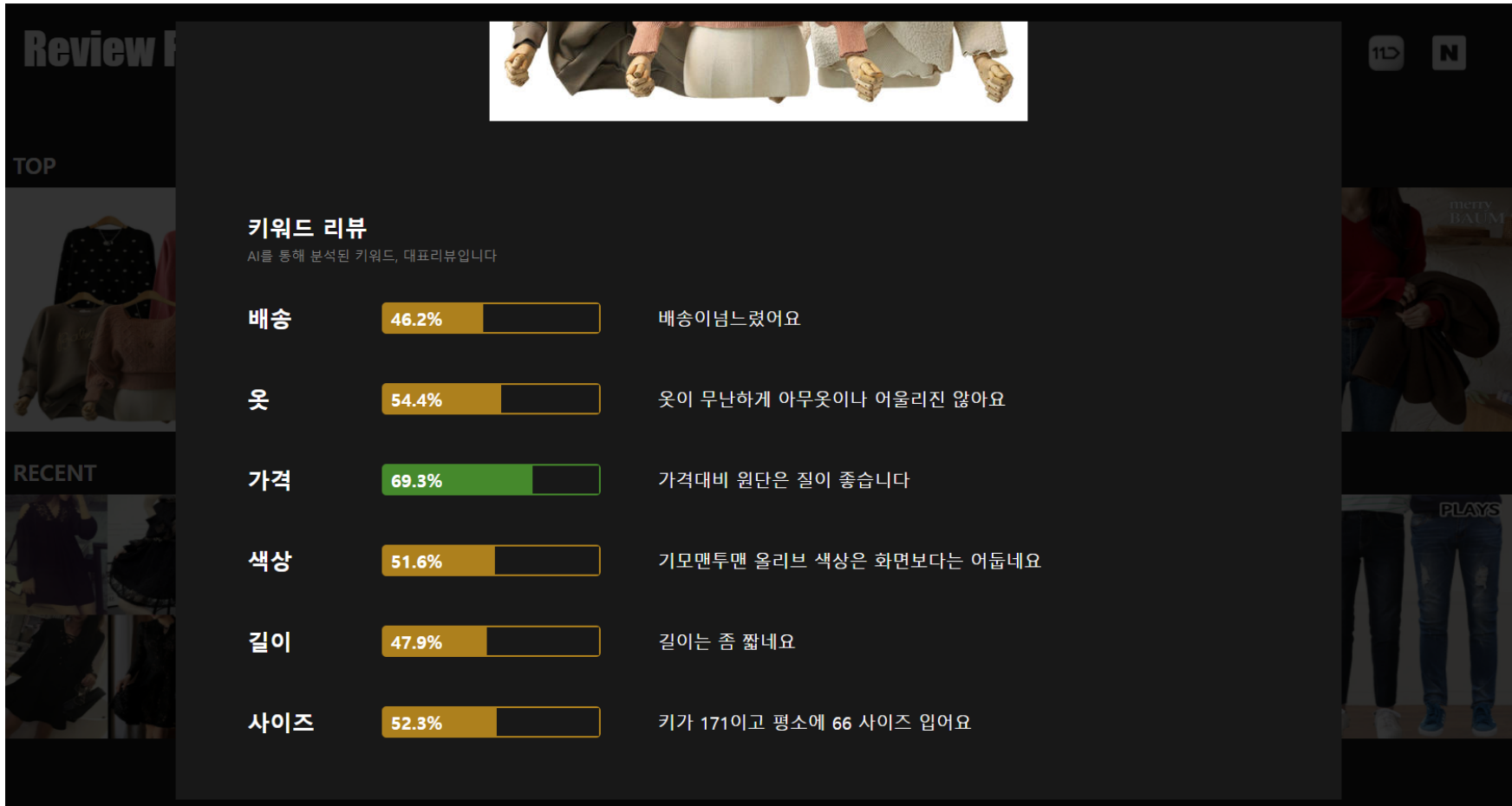


| 화면 설계도



기능 명	원하는 상품 선택 및 URL / 카테고리 검색
기능설명	홈페이지의 메인 화면으로, 사용자로 부터 URL / 카테고리 검색을 입력 받음 1) URL 입력 시 - 해당 상품에 대한 분석 결과 창을 노출 2) 카테고리 입력 시 - 카테고리에 해당 하는 상품들을 많이 검색된 순으로 , 최근 검색 순 으로 구분하여 노출
처리내용	사용자로부터, 입력을 받으면, 해당 입력이 URL 인지 카테고리인지에 따라, 다르게 실행됩니다. URL 일 경우, 크롤링을 통해 쇼핑 물로부터 리뷰 데이터를 저장하며, 이를 분석 및 출력합니다. 해당 내용이 카테고리일 경우, 각 카테고리에 해당하는 상품의 정보를 출력합니다.

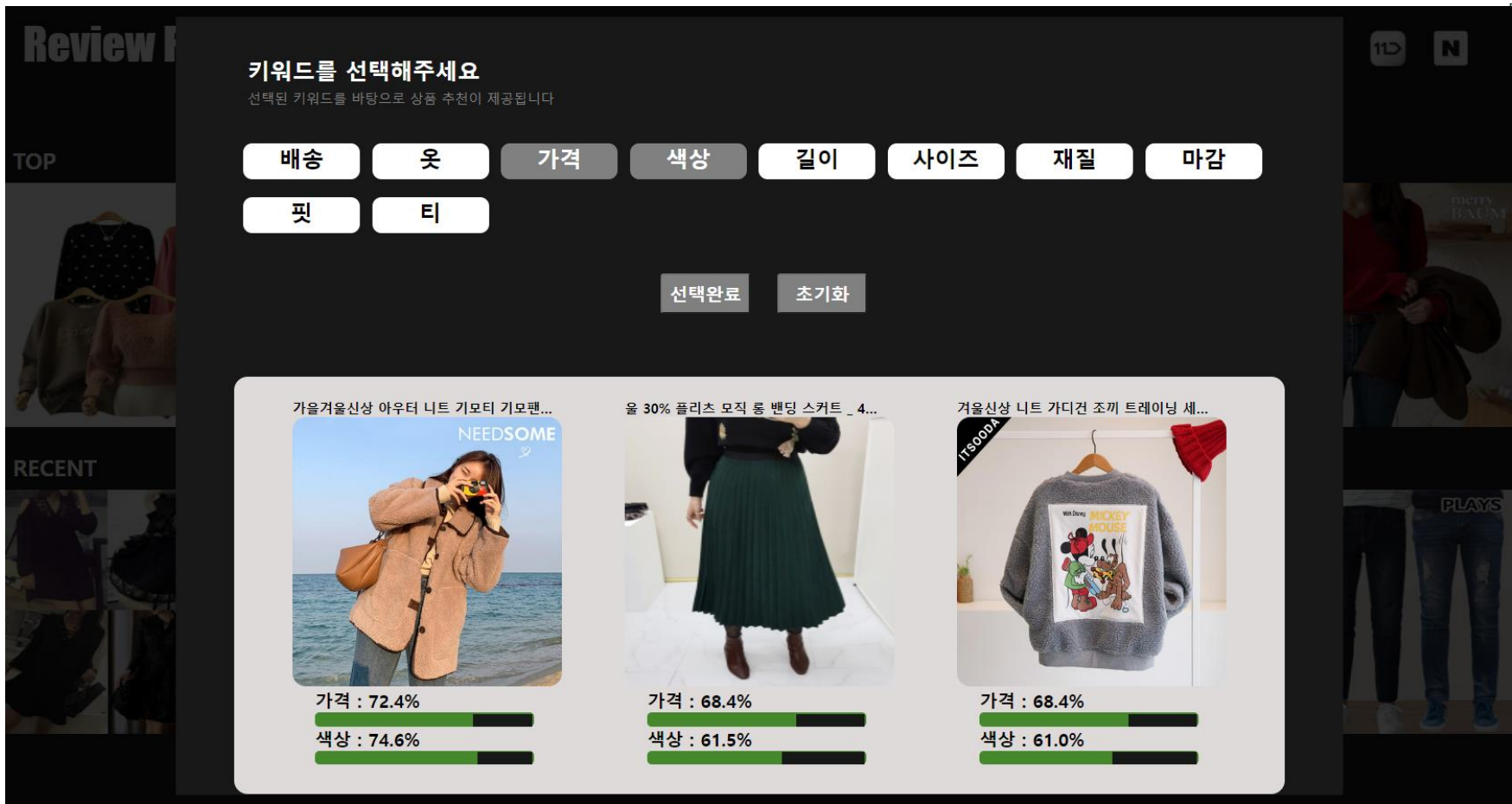
| 화면 설계도

	<table> <tr> <th>기능 명</th><td>(상품 클릭) 상품 상세 정보 [상품정보 및 키워드 별 긍정 부정 비율]</td></tr> <tr> <th>기능설명</th><td>상품 클릭 시, 상품에 대한 상세 설명 및 분석 내용을 출력 (키워드 추출)</td></tr> <tr> <th>처리내용</th><td>상품 검색 및 클릭 시, 상품에 대한 설명 및 분석 결과를 데이터 베이스로 부터 가져와 사용자에게 출력합니다. 상품에 대한 대표 키워드를 나타내고, 각 키워드 별, 상품에 대한 긍정, 부정 비율 및 대표 문장을 요약하여 추출합니다.</td></tr> </table>	기능 명	(상품 클릭) 상품 상세 정보 [상품정보 및 키워드 별 긍정 부정 비율]	기능설명	상품 클릭 시, 상품에 대한 상세 설명 및 분석 내용을 출력 (키워드 추출)	처리내용	상품 검색 및 클릭 시, 상품에 대한 설명 및 분석 결과를 데이터 베이스로 부터 가져와 사용자에게 출력합니다. 상품에 대한 대표 키워드를 나타내고, 각 키워드 별, 상품에 대한 긍정, 부정 비율 및 대표 문장을 요약하여 추출합니다.
기능 명	(상품 클릭) 상품 상세 정보 [상품정보 및 키워드 별 긍정 부정 비율]						
기능설명	상품 클릭 시, 상품에 대한 상세 설명 및 분석 내용을 출력 (키워드 추출)						
처리내용	상품 검색 및 클릭 시, 상품에 대한 설명 및 분석 결과를 데이터 베이스로 부터 가져와 사용자에게 출력합니다. 상품에 대한 대표 키워드를 나타내고, 각 키워드 별, 상품에 대한 긍정, 부정 비율 및 대표 문장을 요약하여 추출합니다.						

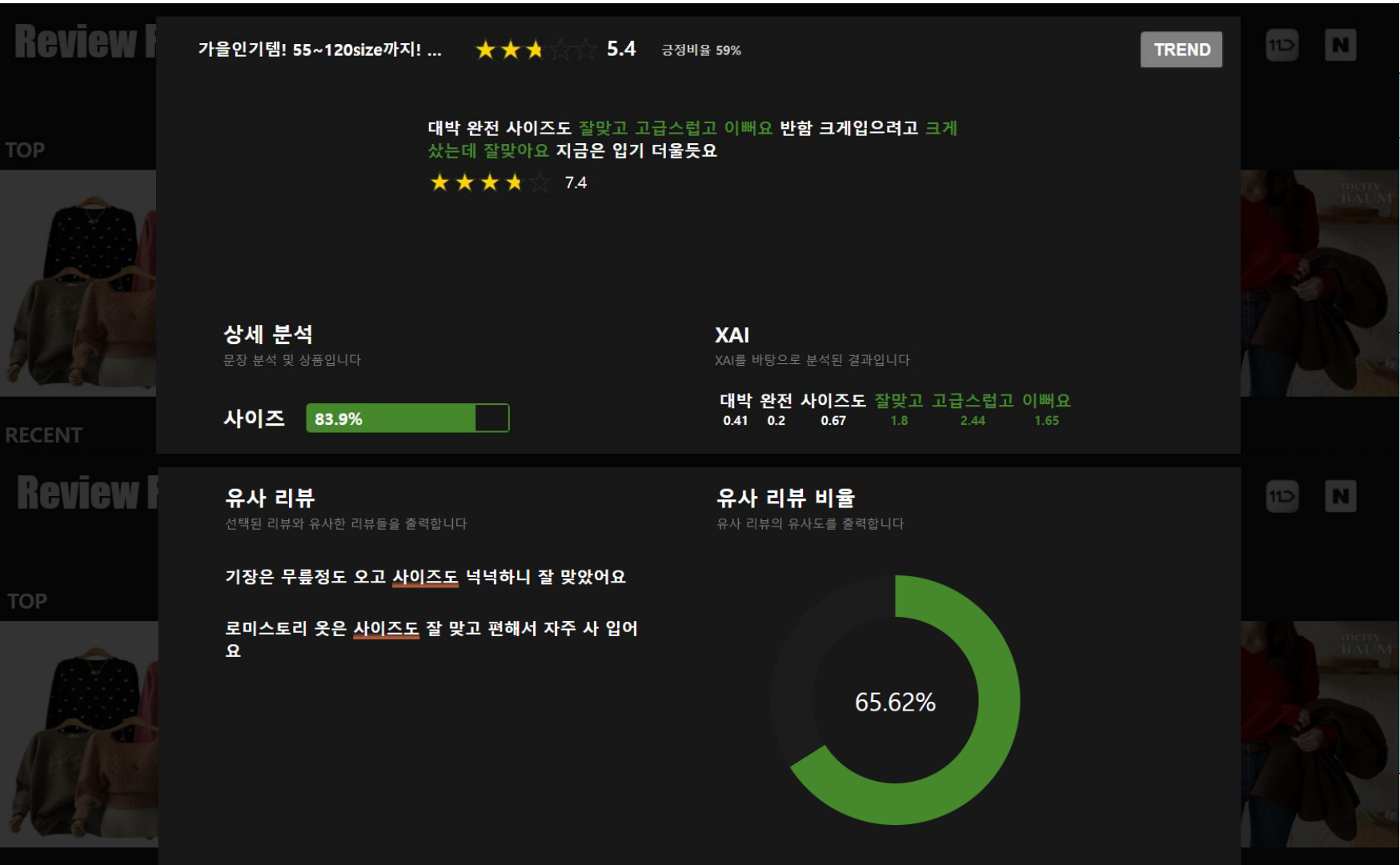
| 화면 설계도

 The screenshot displays a product review section with two columns: 'Positive Reviews' (긍정 리뷰) and 'Negative Reviews' (부정 리뷰). Each review includes a star rating, a short text snippet, and a list of tags. The interface also features a 'Review' (리뷰) button and a 'Filter' (필터) button.	기능 명	상품 감성분석 (XAI) 및 키워드 분석
기능설명	리뷰 데이터를 분석하여, 긍부정 감성분석 및 각 문장에 키워드를 추출합니다.	
처리내용	감성분석 결과를 바탕으로 해당 리뷰를 절대 평점으로 표현하여 사용자에게 객관적인 평가지표를 제시하고 판단 결과의 근거를 시각적으로 제시하여 리뷰에 대한 사용자의 이해도를 증가시킵니다. 또한 각 문장에서 표현된 키워드를 통해 해당 리뷰가 어떤 키워드에 대한 평가인지 보여줍니다.	

| 화면 설계도

	<table> <tr> <td>기능 명</td><td>상품 추천</td></tr> <tr> <td>기능설명</td><td>원하는 키워드를 선택하고, 관련 키워드 평가가 우수한 상품을 추천합니다.</td></tr> <tr> <td>처리내용</td><td>선택된 키워드들을 모두 고려하여 이에 대한 평가가 좋은 상품을 추천합니다.</td></tr> </table>	기능 명	상품 추천	기능설명	원하는 키워드를 선택하고, 관련 키워드 평가가 우수한 상품을 추천합니다.	처리내용	선택된 키워드들을 모두 고려하여 이에 대한 평가가 좋은 상품을 추천합니다.
기능 명	상품 추천						
기능설명	원하는 키워드를 선택하고, 관련 키워드 평가가 우수한 상품을 추천합니다.						
처리내용	선택된 키워드들을 모두 고려하여 이에 대한 평가가 좋은 상품을 추천합니다.						

| 화면 설계도



기능 명	상세 리뷰 클릭 시
기능설명	자세한 정보를 얻고 싶은 리뷰를 클릭 시, 해당 리뷰에 대한 분석 결과를 출력합니다.
처리내용	리뷰 클릭 시, 해당 리뷰에 대한 감성분석 결과 (XAI 를 통한 하이라이팅 및 수치적 표현) 출력 및 키워드 평가를 기본적으로 보여주며 더 나아가 리뷰의 각 문장 중 원하는 특성을 언급하는 문장을 선택할 시, 그 문장에 대한 유사도 분석, 유사 리뷰 등을 출력합니다.

| 화면 설계도

상품 추천

선택한 리뷰를 바탕으로 추천된 상품입니다

신상 롱 원피스 기모 반팔 세트 뷔스티에...

HIFASHIONPURPLE

재질 : 69.3%

10%십일절SALE! BEST100종겨울신상! ...

LUCKY 567

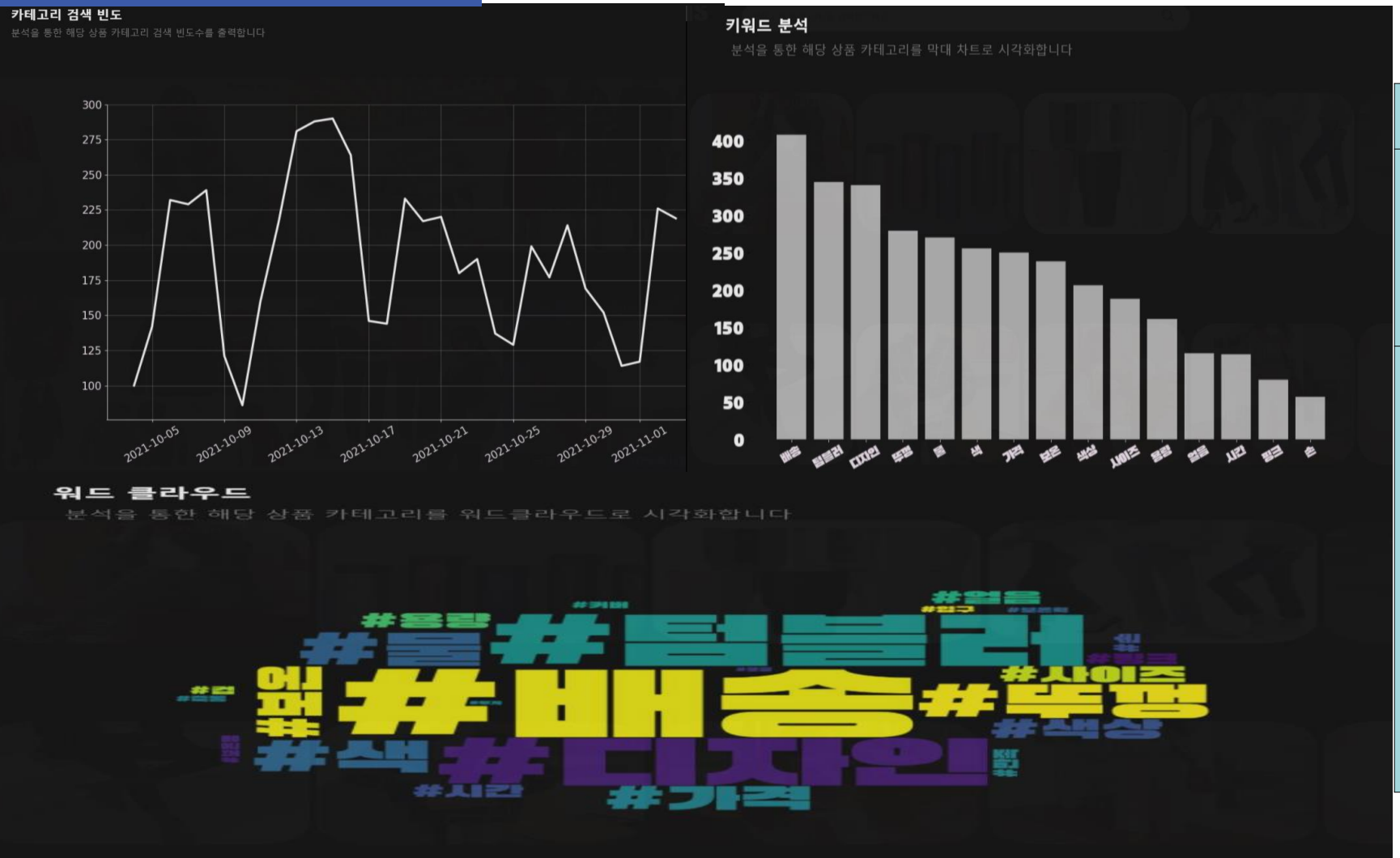
재질 : 68.3%

양 브이넥 골지 슬림 니트 V넥 니트

재질 : 65.0%

기능 명	클릭된 리뷰 기반 상품추천
기능설명	클릭한 리뷰를 분석하여, 상품을 추천합니다.
처리내용	해당 리뷰에서 언급된 키워드들을 모두 고려하여 이에 대한 평가가 좋은 상품을 추천합니다.

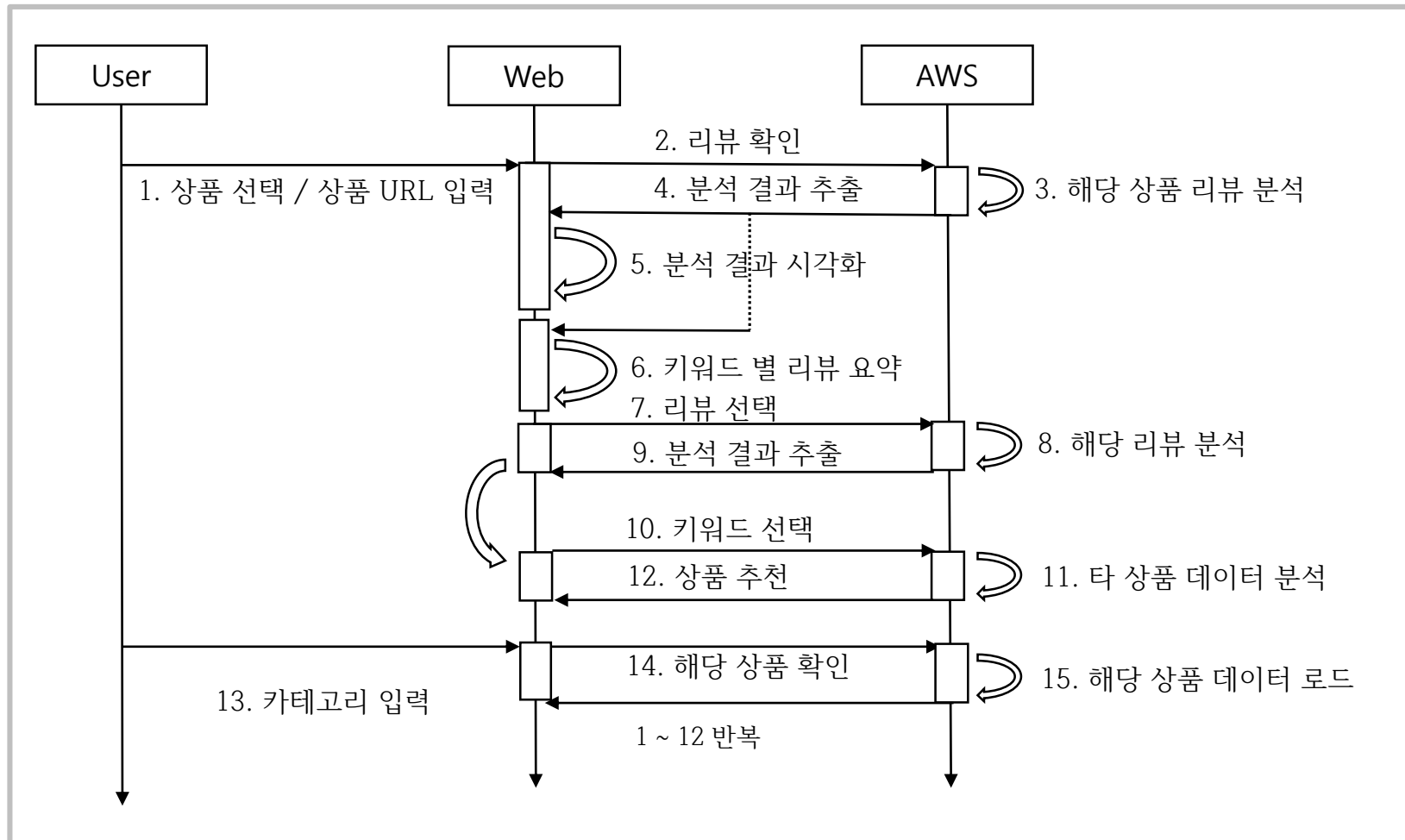
| 화면 설계도



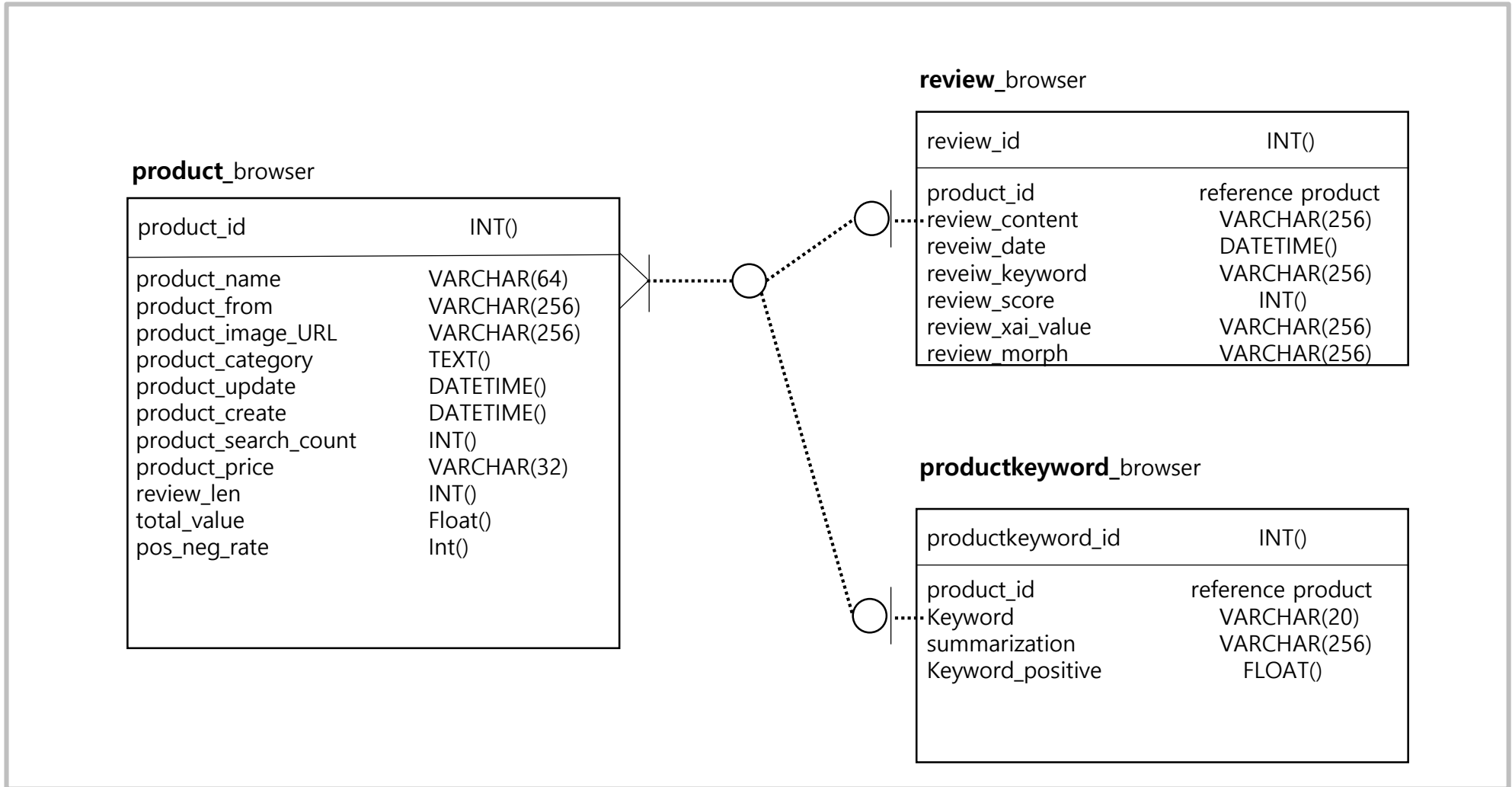
기능 명	상품 트렌드 분석
기능설명	해당 카테고리에 대한 트렌드 분석 결과를 시각화 하여 출력합니다.
처리내용	입력 받은 카테고리에 해당하는 트렌드 분석을 합니다. 해당 카테고리에 대한 시간 흐름에 따른 관심도, 키워드 워드 클라우드, 빈도수 분석, 트리 맵 분석을 시각화 하여 나타냅니다.

| 기능 처리도(기능 흐름도)

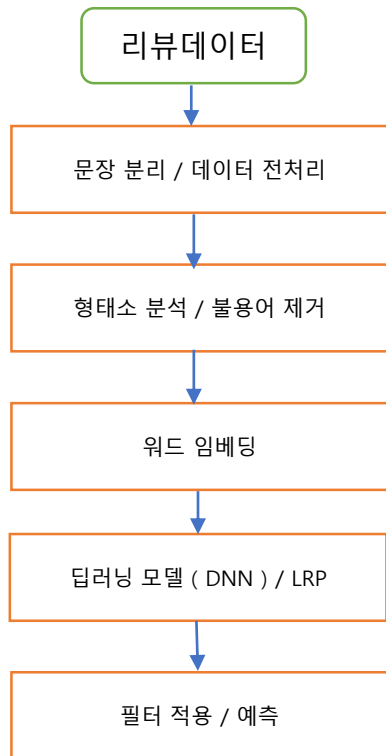
▶ 리뷰 데이터 분석 및 사용자 경험 분석



| 엔티티관계도 – ERD



감성분석 (XAI) 알고리즘



○ 알고리즘 시나리오

1. 사용자로 부터 URL을 입력 받고, 해당 URL에 있는 리뷰데이터를 입력 크롤링하여 가져옵니다.
2. 리뷰데이터의 문장단위로 분리하고, 데이터 전처리를 진행합니다. 데이터 전처리로는, 특수기호, 자음, 모음, 연속된 공백 등을 삭제합니다.
3. Konlpy Mecab 형태소 분석기를 통해서 형태소 단위로 쪼개며, 이후 불용어에 해당하는 형태소를 제거합니다. 불용어로 설정한 형태소는 조사, 어말, 접두사, 접미사, 부호, Unknown 입니다.
4. 이후 긍정에서 해당 형태소 출현한 비율, 부정에서 출현한 비율의 차를 통해 워드 임베딩 합니다.
5. 사전에 DNN 모델을 통해서 학습된 가중치, 편향 값을 통해 LRP, 즉 각 Input layer 가 output layer에 끼치는 영향 값을 계산합니다.
6. 자체 개발한 필터 알고리즘을 적용하여 값을 보정하고, 이를 이용해 해당 리뷰데이터가 긍정인지 부정인지 예측합니다.

| 알고리즘 상세 설명서

○ 감성분석 XAI 상세 설명서

DNN 모델의 경우, weights 및 bias 의 합으로 결과를 나타내게 됩니다. 이러한 값들을 각각 따로 계산하여 각각의 컬럼이 미치는 영향 값을 계산합니다. (현재 모델에서는, 양수 값이 클 수록, 긍정 영향도가 큰 단어, 음수 값이 클 수록, 부정 영향도가 큰 단어)

이후 개발한 필터 알고리즘을 이용하여, 값을 보정 합니다.

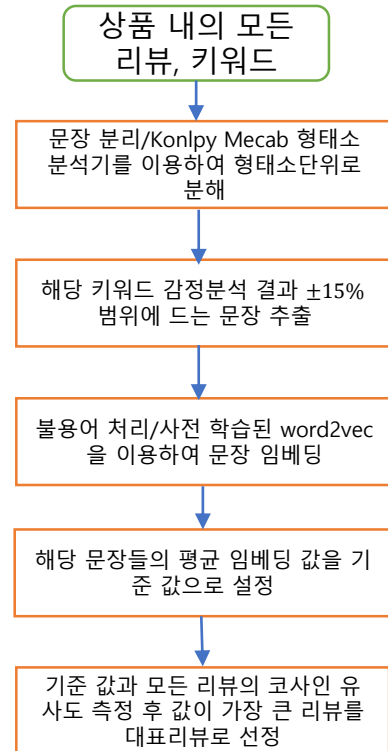
영향 값이 일정 값 보다 크거나, 혹은 일정 값보다 작을 시, 다른 단어와 차별되게 시각화
(이미지 예시 : $0.7 \geq x$: 긍정, $-0.7 \leq x$: 부정)



출처 : Layer-Wise Relevance Propagation: An Overview, Grégoire Montavon, Alexander Binder, Sebastian Lapuschkin, Wojciech Samek, and Klaus-Robert Müller

| 알고리즘 명세서 (2)

리뷰요약 알고리즘



○ 알고리즘 시나리오

1. 해당 상품의 모든 리뷰를 불러옵니다.
2. 모든 리뷰를 문장으로 분리한 후 문장들을 KoNLpy Mecab 형태소 분석기를 이용하여, 형태소 단위로 쪼갭니다.
3. 각 키워드의 감정분석 결과 $\pm 15\%$ 범위에 드는 문장을 추출해 줍니다.
4. 불용어, 특수문자, 자음 혹은 모음으로만 이루어진 단어를 제거하는 정제 과정을 거칩니다.
5. 사전 학습된 word2vec 임베딩 데이터를 이용하여, 각 문장을 임베딩 해줍니다.
5. 임베딩 결과 값을 평균 내어 해당 값을 기준 값으로 설정합니다.
6. 기준 값과 모든 리뷰의 코사인 유사도를 측정한 후 유사도가 가장 큰 리뷰를 대표리뷰로 선정합니다.

| 알고리즘 상세 설명서 (2)

○ 리뷰 요약 상세 설명서

모든 리뷰를 문장 별로 분리한 후 Mecab을 통해 형태소를 분리해 줍니다. 이후 모든 문장의 감정분석 결과를 바탕으로 해당 키워드 감정분석 결과 수치의 15% 에 드는 문장들을 추출합니다. 이후 문장들을 불용어 처리, 의미를 반영한 임베딩을 위해 54만개의 데이터를 통해 사전 학습된 word2vec를 사용합니다. 마지막으로 각 문장들의 임베딩 값을 통해 평균 임베딩 값을 구하고 해당 값과 모든 리뷰의 코사인 유사도를 측정한 후 유사도가 가장 큰 리뷰를 대표리뷰로 선정합니다.

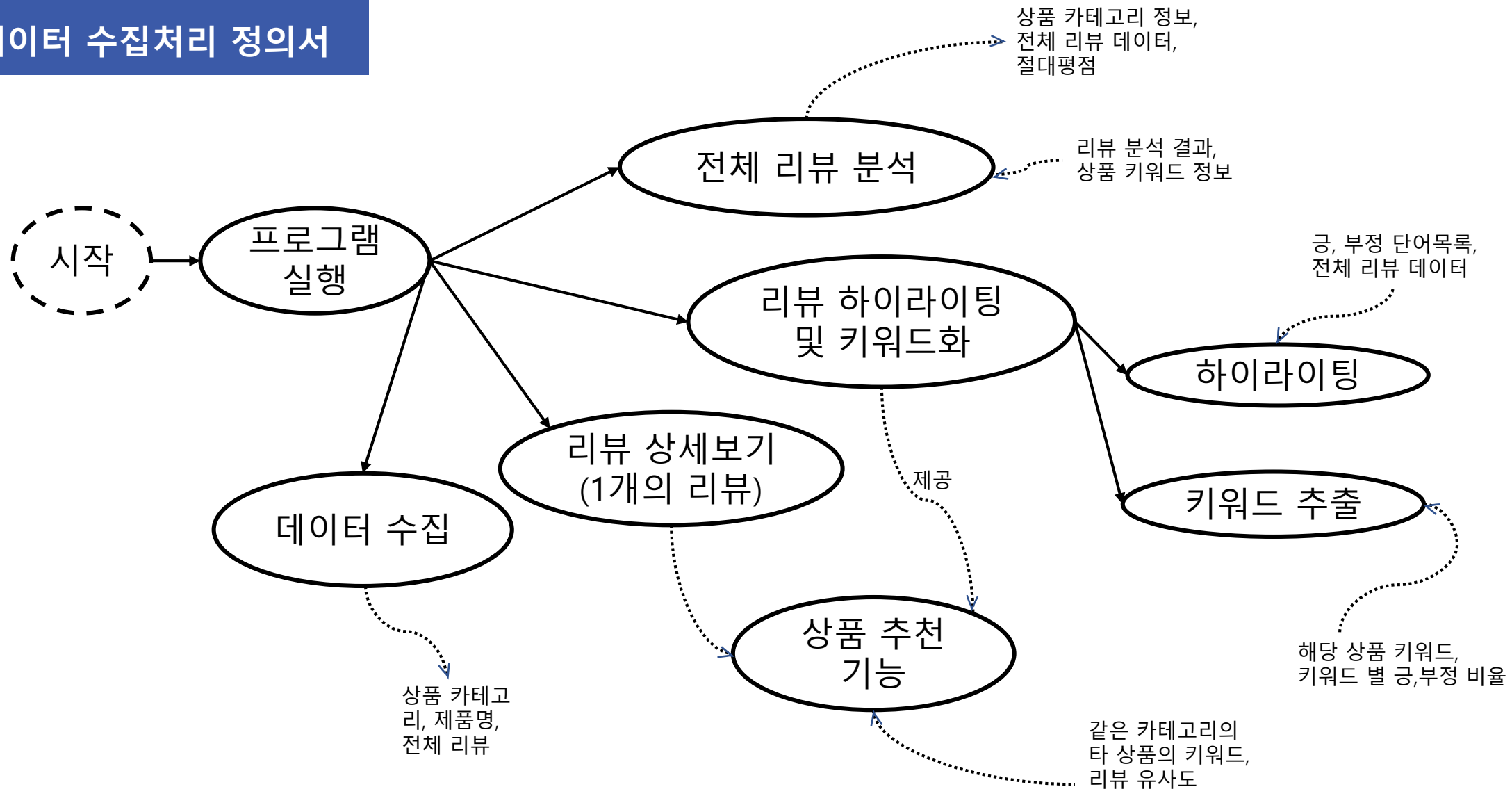
[코사인 유사도 측정]

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	1.000000	0.573450	0.523651	0.672960	0.407591	0.255949	0.451977	0.713510	0.768952	0.471583	0.534051	0.531270	0.440227	0.588177	0.793882
1	0.573450	1.000000	0.512489	0.581665	0.563554	0.204514	0.477717	0.503100	0.517293	0.571298	0.546667	0.526721	0.502760	0.560529	0.722093
2	0.523651	0.512489	1.000000	0.529708	0.242195	0.151974	0.506260	0.351387	0.395559	0.377230	0.658600	0.384407	0.618111	0.519320	0.627633
3	0.672960	0.581665	0.529708	1.000000	0.527013	0.472494	0.505069	0.676737	0.707698	0.519895	0.557922	0.625457	0.654804	0.656427	0.859511
4	0.407591	0.563554	0.242195	0.527013	1.000000	0.321929	0.540734	0.493962	0.398075	0.519353	0.448008	0.648474	0.434739	0.365370	0.662890
5	0.255949	0.204514	0.151974	0.472494	0.321929	1.000000	0.148816	0.288716	0.281369	0.261214	0.066093	0.269513	0.316322	0.186225	0.436861
6	0.451977	0.477717	0.506260	0.505069	0.540734	0.148816	1.000000	0.397637	0.576885	0.445690	0.628667	0.718593	0.681653	0.613435	0.735758
7	0.713510	0.503100	0.351387	0.676737	0.493962	0.288716	0.397637	1.000000	0.708565	0.463451	0.390073	0.625928	0.365267	0.518771	0.769391
8	0.768952	0.517293	0.395559	0.707698	0.398075	0.261369	0.576885	0.708565	1.000000	0.451716	0.534219	0.614375	0.553311	0.742723	0.835260
9	0.471583	0.571298	0.377230	0.519895	0.519353	0.261214	0.445690	0.463451	0.451716	1.000000	0.439483	0.583438	0.409263	0.419056	0.662326
10	0.534051	0.546667	0.658600	0.557922	0.448008	0.066093	0.628667	0.390073	0.534219	0.439483	1.000000	0.517652	0.629887	0.676953	0.714708
11	0.531270	0.526721	0.384407	0.625457	0.648474	0.269513	0.718593	0.625928	0.614375	0.583438	0.517652	1.000000	0.636816	0.485960	0.814563
12	0.440227	0.502760	0.618111	0.654804	0.434739	0.316322	0.681653	0.365267	0.553311	0.409263	0.629887	0.636816	1.000000	0.621367	0.760617
13	0.588177	0.560529	0.519320	0.656427	0.365370	0.186225	0.613435	0.518771	0.742723	0.419056	0.676953	0.485960	0.621367	1.000000	0.768959
14	0.793882	0.722093	0.627633	0.859511	0.662890	0.436861	0.735758	0.769391	0.835260	0.662326	0.714708	0.814563	0.760617	0.768959	1.000000

[상품: 마스크]

배송 : 배송도 빠릅니다
 마스크 : 마스크질이 너무 좋아요
 안쪽 : 안쪽이 부드럽네요
 귀 : 귀도 안 아프고 좋아요
 얼굴 : 얼굴에 잘 맞아요

| 데이터 수집처리 정의서



| 프로그램 - 목록

기능 분류	기능번호	기능 명
PRI	PRI-01	상품 출력
URL	URL-01	URL 입력
CATE	CATE-01	카테고리 입력
	CATE-01-01	카테고리 입력 - 해당 상품 카테고리 별 TOP10/최근10 출력
PRO	PRO-01	상품 선택 (혹은 상품 URL 입력 후 상품 확인)
	PRO-01-01	상품 선택-리뷰 확인
	PRO-01-01-01	상품 선택-리뷰 확인-리뷰 하이라이팅 및 키워드 별 분석 확인
	PRO-01-01-02	상품 선택-리뷰 확인-리뷰 키워드 요약 확인
KEY	KEY-01	키워드 선택
	KEY-01-01	키워드 선택-키워드 별 리뷰 확인
	KEY-01-02	키워드 선택-키워드 별 상품 추천
REV	REV-01	리뷰 클릭
	REV-01-01	리뷰 클릭 - 해당 리뷰 상세 분석 결과 확인
	REV-01-02	리뷰 클릭 - 해당 리뷰 바탕 상품 추천
TRE	TRE-01	트렌드 분석 확인

| 테이블 정의서 – ERD (1)

product	
pk	Product_id
	Product_name
	Product_from
	Product_image_URL
	Product_category
	Product_update
	Product_create
	Product_search_count
	Product_price
	Review_len
	Total_value
	Pos_neg_rate

review	
pk	Review_id
	Product_id
	Review_content
	Review_date
	Review_keyword
	Review_score
	Review_xai_value
	Review_morph

Product_keyword	
pk	Product_keyword_id
	Product_id
	Keyword
	Summarization
	Keyword_positive

| 테이블 정의서 – ERD (2)

항목명	Type	필수 / 선택	값 목록	활성여부	설명
Product_id	Int	필수		활성	상품 ID
Product_url	Char	필수		활성	상품 URL
Product_name	Char	필수		활성	상품 이름
Product_num	Int	필수		활성	상품 번호
Categories	Char	필수		활성	상품 카테고리
Img_src	Char	필수		활성	상품 이미지 src
Price	Char	필수		활성	상품 가격
Review_len	Int	필수		활성	리뷰 개수
Created	Date	필수		활성	생성 날짜
Updated	Date	필수		활성	업데이트 날짜
Total_value	Float	필수		활성	평점
Pos_neg_rate	Float	필수		활성	긍부정 비율
Search_count	Int	필수		활성	검색 빈도

항목명	Type	필수 / 선택	값 목록	활성여부	설명
Review_id	Int	필수		활성	리뷰 ID
Review	Char	필수		활성	리뷰 내용
Score	Float	필수		활성	리뷰 평점
keywords	Char	필수		활성	리뷰 키워드
Keywords_len	Int	필수		활성	키워드 개수
Morph	Char	필수		활성	리뷰 (형태소 단위)
Xai_value	Char	필수		활성	XAI값(형태소 단위)
Product_keyword_id	Int	필수		활성	Product_keyword_id
Keyword	Char	필수		활성	키워드
Summarization	Char	필수		활성	요약 내용
Keyword_positive	Float	필수		활성	키워드 긍부정비율
Review_date	Date	필수		활성	댓글입력 날짜

| 핵심소스코드(1-1)

Embedding 방법 (감성분석 / XAI)

```
negative_sentences = train_data[train_data['label'] == 0]['stem_reviews']
positive_sentences = train_data[train_data['label'] == 1]['stem_reviews']
negative = []
for li in negative_sentences:
    negative.extend(li)

positive = []
for li in positive_sentences:
    positive.extend(li)

negative_sentences = pd.DataFrame(pd.DataFrame(negative).value_counts())
positive_sentences = pd.DataFrame(pd.DataFrame(positive).value_counts())
con = pd.concat([negative_sentences, positive_sentences], axis=1)
con.columns = ['negative', 'positive']
con = con.dropna()
con['negative_value'] = con[['negative', 'positive']].apply(lambda x : x[0]/(x[0]+x[1]) *-1, axis=1)
con['positive_value'] = con[['negative', 'positive']].apply(lambda x : x[1]/(x[0]+x[1]), axis=1)
con['total_value'] = con['negative_value'] + con['positive_value']
con['total'] = con['negative']+con['positive']
con = con[con['total'] > 10]
```

해당 단어가, 긍정, 부정에서 나오는 횟수를 Count하여, 임베딩 진행

임베딩 방법은 우측과 같음

○ Embedding

- 각각의 단어가 긍정인 문장 및 부정에 나오는 문장의 빈도 수를 이용하여 계산

$$PositiveValue = \frac{PositiveCount}{TotalCount}$$

$$NegativeValue = \frac{NegativeCount}{TotalCount}$$

$$TotalValue = PositiveValue - NegativeValue$$

	0	negative	positive	negative_value	positive_value	total_value
강추		109.0	3431.0	-0.030791	0.969209	0.938418
개취		20.0	20.0	-0.500000	0.500000	0.000000
더럽		583.0	27.0	-0.955738	0.044262	-0.911475
비추		4036.0	144.0	-0.965550	0.034450	-0.931100
좋아요		10.0	364.0	-0.026738	0.973262	0.946524


```
def sss(text):
```

```
    text = re.sub(r" {2,}", " ", hangul.sub(' ', text))
```

정규표현식을 이용해 연속된 공백 제거

```
    end_char = ['요', '다', '쥬']
```

```
    avoid_char = ['보다', '하려다', '하다', '려다']
```

문장 끝맺음에 자주 등장하는 단어 및 예외 단어 처리

```
    special_char = ['느림']
```

```
    new_sentences = []
```

```
    ts = text.split()
```

```
    start, end, flag = 0, 0, 0
```

```
    for i in range(len(ts)):
```

```
        if (len(ts[i]) >= 2 and ts[i][-1] in end_char and ts[i][-2:] not in avoid_char)\
            or ('ETN' in mecab.pos(ts[i])[-1][1]) or (ts[i][-2:] in special_char): # and mecab.pos(ts[i])[-1][1] != 'NNG'
```

“뒀”, “함” 등 명사형으로 끝나는 단어

```
            end = i
```

```
            new_sentences.append(' '.join(ts[start:end + 1]).strip())
```

```
            start = end + 1
```

```
            flag = 1
```

```
        else:
```

```
            if i == len(ts)-1:
```

```
                new_sentences.append(' '.join(ts[start:]).strip())
```

```
    if not flag and len(new_sentences) == 0:
```

```
        new_sentences.append(text)
```

```
    return new_sentences
```

| 핵심소스코드(1-3)

입력된 문장을 감성분석 모델을 통해 분석 (XAI)

```

after_preprocess = re.sub(r" {2,}", " ", hangul.sub(' ', sentence))
after_preprocess = after_preprocess.strip()
tmp = [text[0] for text in mecab.pos(after_preprocess) if
        text[1][0] != 'E' and text[1][0] != 'S' and (text[1] == 'XR' or text[1][0] != 'X') and text[1][0] != 'J' and text[
            1] != 'UNKNOWN' and text[0] != '.' and text[0] != '웃']
value = [float(con[con[:], 0] == word, 1)) if word in word_index else 0 for word in tmp]
if len(value) >= 20:
    value = value[:20]
else:
    value = np.pad(value, (0, 20 - len(value)), 'constant')

```

형태소 분석 이후, 불용어 제거

딥러닝 모델에 입력하기 위해 padding 및 초과 삭제

```

consider_relu = [0 if h < 0 else 1 for h in np.dot(value, weights[0]) + weights[1]]
arr = [*map(np.sum, [[h * weights[2][index] + weights[3] / (20 * 10) for index, h in enumerate(
    [0 if consider_relu[index] == 0 else value[i] * weights[0][i][index] + weights[1][index] / 20 for index in
    range(10)]] for i in range(20)]])]

```

LRP 를 통한 각 Input layer마다 output layer에 끼치는 영향 값 계산

```

before_text = sentence.split(' ')
arr_num, values = 0, []
for before_mecab in before_text:
    value_tmp = 0
    for k in range(len(mecab.morphs(before_mecab))):
        if arr_num + k >= 20: break
        value_tmp = value_tmp + arr[arr_num + k]
    else:
        k=0
    arr_num = arr_num + k + 1
    values.append(round(value_tmp, 3))

```

형태소 분석기로 쪼갠 형태소를 다시 복구

| 핵심소스코드(1-4)

입력된 문장을 감성분석 모델을 통해 분석 (XAI)

```

def make_score(x):
    if x < -2.16:
        return -2
    elif x >= 1.84:
        return 2
    else:
        return x+0.16

def make_score2(x):
    return x/4 +0.5

make_score2(make_score(sum(xai_value)/len(xai_value)))

if len(values) < 3:
    if max(values) > min(values)*-1:
        tmp = [max(values) for i in range(len(values))]
    else:
        tmp = [min(values) for i in range(len(values))]
    return before_text, [*map(lambda x: round(x, 2), tmp)], positive_or_negative(value), values

tmp = []
for i in range(len(values)):
    if i ==0:
        big = max(values[i],values[i + 1],values[i + 2])
        small = min(values[i],values[i + 1],values[i + 2])
        if big >= small*-1:
            tmp.append(big)
        else:
            tmp.append(small)
    elif i== len(values)-1:
        big = max(values[i],values[i - 1],values[i - 2])
        small = min(values[i],values[i - 1],values[i - 2])
        if big >= small*-1:
            tmp.append(big)
        else:
            tmp.append(small)
    else:
        big = max(values[i],values[i - 1],values[i +1])
        small = min(values[i],values[i - 1],values[i +1])
        if big >= small*-1:
            tmp.append(big)
        else:
            tmp.append(small)

return before_text, [*map(lambda x: round(x, 2), tmp)], positive_or_negative(value), values

```

```

def make_score(x):
    if x < -2.16:
        return -2
    elif x >= 1.84:
        return 2
    else:
        return x+0.16

def make_score2(x):
    return x/4 +0.5

make_score2(make_score(sum(xai_value)/len(xai_value)))

```

자체 개발한 필터를 통해 값을 보정 및 활성화 함수를 통한 예측

```

vocab_sorted = count_noun(nouns) # 명사 키워드
check_vocab = vocab_sorted[:30] # 20개 출력
# JKS, JX_중요조사들
josa = ['JKS', 'JX'] # 품사 중 조사에 대한 표현 저장 #+JKO, JKB, JKG, JKV, JKC, JC
word_next_josa = {w: 0 for w, k in vocab_sorted} # 단어 뒤에 조사가 붙는지에 대한 count를 저장하기 위한 딕셔너리
for i in range(len(review_data_list)):
    for word, value in vocab_sorted: # 저장된 단어들 호출
        for idx in range(len(review_data_list[i])):
            if word == review_data_list[i][idx][0]:
                if idx + 1 < len(review_data_list[i]) and review_data_list[i][idx + 1][
                    1] in josa: # 해당 단어의 다음에 조사가 나온다면
                    word_next_josa[word] += 1 # count 해줌

```



```

word_josa_count = sorted(word_next_josa.items(), key=lambda x: x[1], reverse=True)[:20] # count를 기준으로 sort 20개
keyword_before = []
for i in range(len(word_josa_count)):
    for j in range(len(check_vocab)):
        if word_josa_count[i][0] == check_vocab[j][0] and word_josa_count[i][
            1] > 1: # 그냥 가장 많이 나온 단어들과 다음 단어가 조사가 나오는 단어들 중 을 선택
            keyword_before.append(word_josa_count[i][0])

```

```
# 유사어 처리 및 불용어 처리(상품명, 제품, 상품)
```

```
similar_word = []
keyword = []
for key in keyword_before:
    if key not in similar_word and key not in useless_NNG:
        keyword.append(key)
        try:
            result = word2vec_model.wv.most_similar(key)
            r = [w for w, v in result] # 54만개의 데이터로 사전 학습된 word2vec을 통
                                     해 유사어 측정
            similar_word.extend(r)
        except KeyError as e:
            pass

return keyword, vocab_sorted
```

```
# 사전 학습된 word2vec 모델 load 후 각 단어에 임베딩값 부여
word2vec_model = gensim.models.Word2Vec.load('word2vec_by_mecab.model')
def get_vector(word):
    if word in word2vec_model:
        return word2vec_model[word]
    else:
        return None
```

mecab 형태소 분석기를 사용한
pretrained word2vec load

```
for i in range(len(review_and_word[word])):
    if float(keyword_pos_neg_ratio) - 15 <= review_and_word[word][i][3] <= float(keyword_pos_neg_ratio)+15:
        temp_list.append(review_and_word[word][i])
review_data_word[word] = temp_list
```

키워드 감정분석 결과 수치를 통해 해당 범위에 포
함되는 문장들만 추출

```
# 각 키워드의 평균 문장벡터 생성 후 기존 각 단어별 리스트에 추가
document_embedding_list = {}
for word in review_data_word:
    document_embedding_list[word] = [review_data_word[word][0][1]]
    for i in range(1, len(review_data_word[word])):
        document_embedding_list[word].append(review_data_word[word][i][1])
```

임베딩 값들로만 표현된 리스트를 생성

```
# 각 키워드의 평균 문장벡터 생성 후 기존 각 단어별 리스트에 추가
document_embedding_list = {}
for word in review_data_word:
    doc_doc2vec = np.zeros(100,)
    for i in range(len(review_data_word[word])):
        doc_doc2vec += review_data_word[word][i][1]
    doc2average = doc_doc2vec / len(review_data_word[word])
    review_data_word[word].append(['리뷰들의 평균 벡터값입니다.', doc2average])
```

리뷰들의 평균 임베딩 벡터값을
기존 리뷰 리스트 마지막에 추가해줌

코사인 유사도 계산

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
cosine_similarities = {}
```

```
for word in document_embedding_list:
```

```
    cosine_similarities[word] = cosine_similarity(document_embedding_list[word], document_embedding_list[word])
```

```
    print(cosine_similarities[word].shape)
```

sklearn의 cosine_similarity 모듈 사용

키워드 별 리뷰 요약 출력

```
for word in keyword:
```

```
    idx = list(cosine_similarities[word][-1]).index(sorted(cosine_similarities[word][-1], reverse=True)[1])
```

```
    print('{} : {}'.format(word, review_data_word[word][idx][0]))
```

키워드 별 리뷰들의 임베딩값을 저장한 리스트의 마지막 요소는 평균 임베딩 값이므로 이와 가장 유사한 임베딩값을 가진 리뷰를 대표리뷰라 선정

| 참조 – 개발 환경 및 설명

구분		항목	적용내역
S/W 개발환경	NLP / 머신러닝	Google Colaboratory	Google Colaboratoty를 이용하여, 머신러닝 모델 학습
		Konlpy	한국어 자연어 처리를 위하여, Konlpy (Mecab) 형태소 분석기 이용
		Word2vec	머신러닝 학습을 위해 단어 임베딩으로 word2vec 이용
		Tensorflow – Keras	딥러닝 프레임워크를 이용하여, 모델 학습
	웹사이트	Front-End	HTML, CSS, JavaScript 를 이용한, 웹사이트 Front-End 제작
		Back-End	Python (Django) – 웹 애플리케이션 개발을 위한 프레임워크 이용
H/W 구성장비	서버	클라우드 서버	AWS – 클라우드 서버를 이용하여, 웹사이트 운영

| 참조 – S/W 기능 실사 사진

Review For You

카테고리 혹은 URL을 입력해주세요

1D N

TOP

RECENT

키워드 리뷰

키워드 선택해주세요

배송 옷

가

색상

길이

사이즈

재질

마감

핏

티

선택완료 초기화

가을가을상상 아우터 나트 기모티 기모콘

물 30% 울리츠 보직 중 뽀얀 스커트... 4...

가을상상 나트 가디건 오뜨 드레이싱 세...

가격 : 72.4%

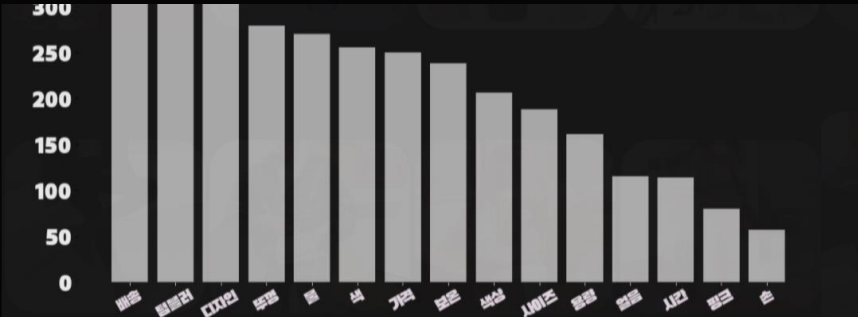
색상 : 74.6%

가격 : 68.4%

색상 : 61.5%

가격 : 68.4%

색상 : 61.0%



Review For You

키워드 리뷰

키워드 선택해주세요

배송 옷

가

색상

길이

사이즈

재질

마감

핏

티

선택완료 초기화

가을가을상상 아우터 나트 기모티 기모콘

물 30% 울리츠 보직 중 뽀얀 스커트... 4...

가을상상 나트 가디건 오뜨 드레이싱 세...

가격 : 72.4%

색상 : 74.6%

가격 : 68.4%

색상 : 61.5%

가격 : 68.4%

색상 : 61.0%



워드 클라우드

분석을 통한 해당 상품 카테고리별 워드클라우드를 시작합니다

#배송 #가디건 #오뜨 드레이싱 세... #가 #색상 #길이 #사이즈 #재질 #마감 #핏 #티

Review For You

키워드 리뷰

키워드 선택해주세요

배송 옷

가

색상

길이

사이즈

재질

마감

핏

티

선택완료 초기화

가을가을상상 아우터 나트 기모티 기모콘

물 30% 울리츠 보직 중 뽀얀 스커트... 4...

가을상상 나트 가디건 오뜨 드레이싱 세...

가격 : 72.4%

색상 : 74.6%

가격 : 68.4%

색상 : 61.5%

가격 : 68.4%

색상 : 61.0%

상품 추천

선택한 리뷰를 바탕으로 추천된 상품입니다

신상 롱 슬리브 기모 세트 베스트 세트...

10%신상일 SALE! BEST100종가을상상!

영 브이넥 솔지 슬리브 니트 V넥 니트

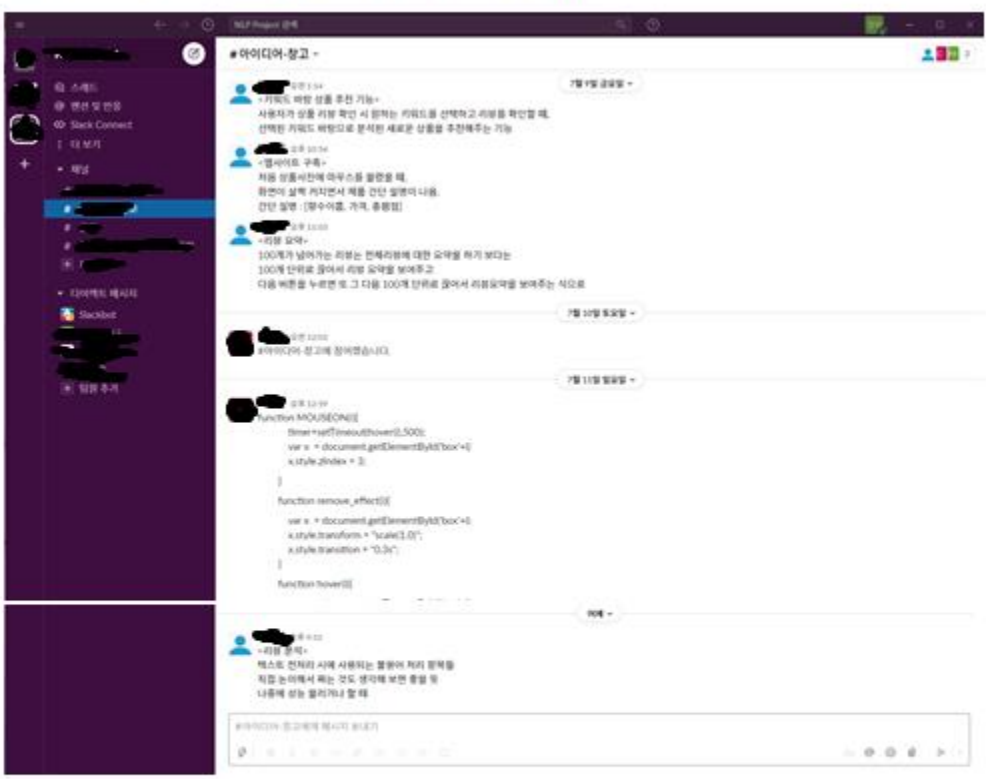
재질 : 69.3%

재질 : 68.3%

재질 : 65.0%

| 참조 - 프로젝트 관리

Slack 을 이용하여, 아이디어 공유



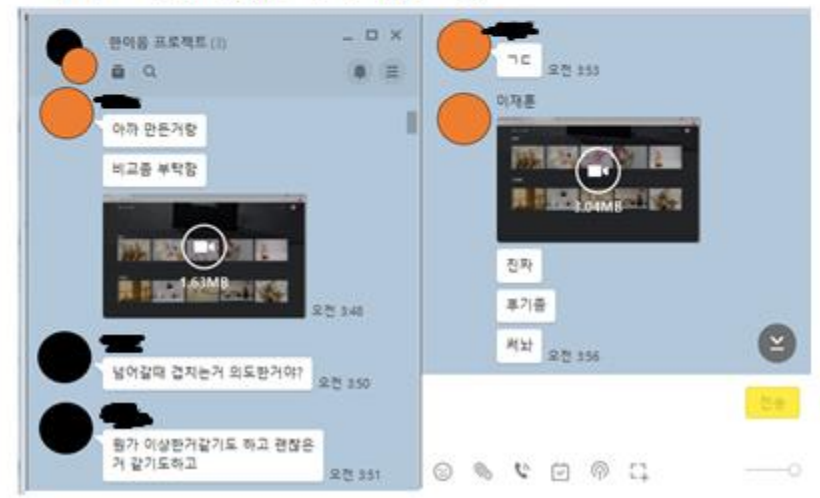
한이음 사이트 타임라인을 이용한 회의록 작성



GitLab을 이용한 코드 공유



카카오톡을 이용한 지속적인 소통



Thank you

